



Enhanced Object Detection and Segmentation in Catering Automation Using YOLO and FastSAM

Makan Baghban 810699182

Amirreza Moharrami 810699260

Supervisor: Dr. Mehdi Tale Masouleh

Mini Project 4

Mechatronics & Robotics

School of Mechanical Engineering

College of Engineering

University of Tehran

July, 2024

Abstract

This report explores the application of deep learning techniques for automating the assembly of custom catering packages. Utilizing the YOLO (You Only Look Once) framework for object detection and the FastSAM model for object segmentation, we developed a system capable of accurately identifying and locating various items within images. The project includes a comprehensive analysis of different YOLO versions, an evaluation of model performance using the mean Average Precision (mAP) metric, and the implementation of data augmentation techniques to enhance dataset robustness. Additionally, we demonstrate the practical integration of object detection and segmentation for improved precision in identifying object shapes and orientations, culminating in a method for grasp point generation essential for robotic manipulation.

Contents

1	Introduction	1
2	YOLO different versions	3
3	mAP score	6
3.1	Steps to Calculate mAP	6
3.1.1	Precision-Recall Curve	6
3.1.2	Intersection over Union (IoU)	7
3.1.3	Computing Precision and Recall	7
3.1.4	Average Precision (AP)	7
3.1.5	Mean Average Precision (mAP)	8
3.2	Practical Considerations	8
3.3	Example Calculation Steps	8
4	Dataset Analysis and Augmentation	10
4.1	Dataset Distribution Analysis	10
4.1.1	Class Balance	10
4.1.2	Dataset Split:	11
4.1.3	The Number of Images and Instances of Each Class	12
4.2	Dataset Health Check	13
4.3	Importance of Augmentation Steps	14
4.4	Augmentation Steps Used in the Dataset	15
4.4.1	Augmentation Steps and Their Uses	15

4.4.2	Preprocessing Steps	16
4.5	Conclusion	16
5	Object detection	17
5.1	Training Phase	17
5.1.1	Data Preparation	17
5.1.2	Colab Code	18
5.1.3	Training Time per Epoch	19
5.2	Testing Phase	20
5.3	Analysis and Discussion	20
5.3.1	Evaluation Metrics	20
5.3.2	Model Validation and Performance	22
5.3.3	Reasons for Errors in Model Predictions	28
5.3.4	Potential Improvements	28
5.4	Conclusion	29
6	Object segmentation	30
6.1	Steps and Code Implementation	30
6.1.1	Mount Google Drive	30
6.1.2	Install Necessary Packages	30
6.1.3	Import Required Libraries	30
6.1.4	Initialize FastSAM Model	31
6.1.5	Function to Perform Segmentation and Save Results	31
6.1.6	Define Directories and Perform Segmentation	33
6.2	Explanation	33
6.3	Results	34
6.4	Conclusion	34
7	Grasp point generation	35
7.1	Steps and Code Implementation	35
7.1.1	Convert FastSAM Segmentations into Contours	35

7.1.2	Use <code>minAreaRect</code> to Obtain Rotated Rectangle	36
7.1.3	Step 3: Calculate the Rotation Angle of the Object	36
7.1.4	Step 4: Identify Grasp Points	37
7.1.5	Step 5: Visualize and Save Grasp Points	38
7.2	Conclusion	39

List of Figures

2.1	Visual Evolution of YOLO	5
4.1	Class Balance	10
4.2	Dataset Split	11
4.3	Dataset Health Check	14
4.4	Augmentation Steps	15
5.1	F1-Confidence Curve	20
5.2	Precision-Confidence Curve	21
5.3	Recall-Confidence Curve	21
5.4	Precision-Recall Curve	22
5.5	Several Results	26
5.6	Training Batch Example	26
5.7	Validation Batch Example	27
5.8	Test Example	27
5.9	Normalized Confusion Matrix	29

Chapter 1

Introduction

In the rapidly evolving field of artificial intelligence (AI), object detection has emerged as a critical technology with broad applications across various industries. Unlike traditional image classification, object detection involves not only identifying objects within an image but also determining their precise locations using bounding boxes. This capability is pivotal for applications such as autonomous vehicles, surveillance systems, and industrial automation.

The evolution of object detection has transitioned from hand-engineered feature extraction methods to advanced deep learning techniques. Modern approaches leverage convolutional neural networks (CNNs) to automatically learn and extract features from raw data, significantly improving accuracy and adaptability.

In this project, we focus on automating the assembly of catering packages by leveraging state-of-the-art object detection and segmentation methods. The YOLO framework, known for its real-time detection capabilities, is employed for object detection tasks. We evaluate different versions of YOLO, highlighting their unique advancements. To enhance our dataset, we employ various data augmentation techniques, ensuring robust model training.

Furthermore, we implement the FastSAM model for object segmentation. Unlike object detection, which provides bounding boxes, segmentation offers pixel-wise masks of objects, allowing for detailed shape analysis. This granularity is crucial for tasks requiring precise manipulation, such as robotic grasping.

The project is divided into several key sections:

1. **Dataset Analysis:** We analyze the distribution of images and instances across different classes within our dataset, emphasizing the importance of even distribution for training.
2. **Model Training and Evaluation:** Detailed steps on training YOLO models, reporting training metrics, and evaluating performance using mAP scores.
3. **Object Detection and Segmentation** Integration of YOLO for detection and FastSAM for segmentation, with practical examples and visualizations.
4. **Grasp Point Generation:** Techniques for determining optimal grasp points using segmented object masks, critical for robotic applications.

This report aims to demonstrate the feasibility and effectiveness of combining object detection and segmentation techniques for automated assembly tasks, providing a foundation for further advancements in industrial automation and AI applications.

Chapter 2

YOLO different versions

YOLOv1 (2015):

- **Initial Release:** YOLOv1 introduced a novel approach to object detection by framing the task as a single regression problem, straight from image pixels to bounding box coordinates and class probabilities. This was a departure from previous methods like Faster R-CNN that used a two-step approach involving region proposal and classification.

YOLOv2 (2016):

- **Batch Normalization:** YOLOv2 added batch normalization to all convolutional layers, stabilizing training, speeding up convergence, and enhancing model accuracy.

- **Anchor Boxes:** Introduced anchor boxes, improving bounding box prediction by using predefined shapes and sizes that the network adjusts for better object fitting.

- **High-Resolution Classifier:** YOLOv2 used a high-resolution classifier by fine-tuning the classifier network on higher-resolution images, improving the accuracy of detection.

YOLOv3 (2018):

- **Multi-Scale Predictions:** YOLOv3 made predictions at three different scales, enabling the detection of objects of varying sizes more accurately.

- **Better Feature Extractor:** It replaced the backbone feature extractor with a new one, called Darknet-53, which is deeper and more efficient, utilizing more layers and residual

connections to enhance feature extraction capabilities.

- **Logistic Class Prediction:** Used logistic regression for class prediction and removed the softmax layer, which allowed for better handling of overlapping class labels.

YOLOv4 (2020):

- **Bag of Freebies and Specials:** YOLOv4 incorporated various training and architectural tricks collectively known as "bag of freebies" and "bag of specials" to improve both training performance and inference speed. This includes techniques like mosaic data augmentation, self-adversarial training, and mish activation function.

- **CSPDarknet53 Backbone:** Introduced CSPDarknet53 as the backbone network, which improved the learning capability and reduced computational cost by splitting feature maps into two parts and merging them through a cross-stage hierarchy.

YOLOv5 (2020 by Ultralytics):

- **Implementation in PyTorch:** YOLOv5 is not an official continuation by the original YOLO authors but a PyTorch implementation by Ultralytics. It made YOLO more accessible and easier to use within the PyTorch framework.

- **Enhanced Performance:** YOLOv5 features further improvements in speed and accuracy, providing different model sizes (small, medium, large, extra-large) to balance the trade-offs between model complexity and performance.

- **Automatic Learning Rate Adjustment:** It includes advanced features such as automatic learning rate adjustment and other hyperparameter optimizations.

YOLOv6 (2022):

- **Efficient Backbone and Neck:** Introduced more efficient backbone and neck structures for better feature extraction and fusion, leading to improved accuracy and speed.

- **Improved Training Techniques:** Utilized more advanced training techniques and data augmentation methods to enhance the robustness and generalization of the model.

YOLOv7 (2022):

- **Extended Model Architecture:** Added new modules for better feature learning, including improved attention mechanisms and advanced pooling techniques.
- **Better Handling of Small Objects:** Focused on improving the detection of small objects within images through better resolution handling and multi-scale feature integration.

YOLOv8 (2023):

- **Optimized Network Design:** Further optimized the network design for real-time performance, balancing between computational efficiency and detection accuracy.
- **Enhanced Post-Processing:** Improved post-processing techniques like non-maximum suppression (NMS) to reduce false positives and improve detection confidence.

YOLOv9 (2024):

- **Advanced Hybrid Models:** Combined the strengths of previous YOLO versions with other advanced object detection techniques, incorporating hybrid models for better accuracy.
- **State-of-the-Art Performance:** Achieved state-of-the-art performance on various benchmarks through extensive experimentation and optimization, setting new standards for real-time object detection.

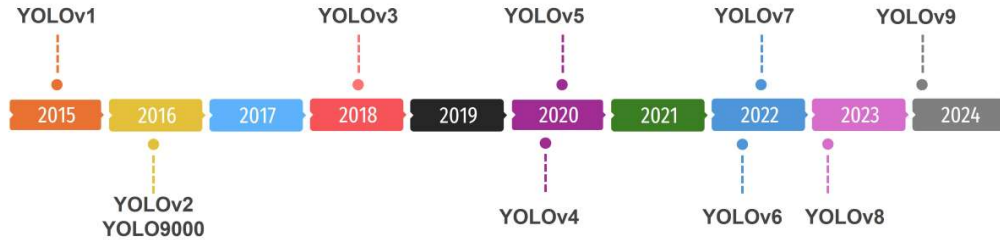


Figure 2.1: Visual Evolution of YOLO

Each version of YOLO brought significant improvements and innovations, making the model more robust, accurate, and faster, thereby broadening its applicability in real-time object detection tasks. The evolution of YOLO showcases ongoing enhancements in deep learning-based object detection.

Chapter 3

mAP score

Overview of mAP:

The mean Average Precision (mAP) score is a key evaluation metric used in object detection tasks to assess the accuracy of the model. It summarizes the precision-recall curve into a single value, providing a comprehensive measure of the model's performance across different classes.

3.1 Steps to Calculate mAP

3.1.1 Precision-Recall Curve

- For each class, the model generates predictions with confidence scores and bounding boxes.
- **Precision** is the ratio of true positive detections to the total number of positive predictions (true positives + false positives).
- **Recall** is the ratio of true positive detections to the total number of actual positive instances (true positives + false negatives).

3.1.2 Intersection over Union (IoU)

To determine whether a prediction is a true positive (TP), the predicted bounding box is compared with the ground truth box using Intersection over Union (IoU). IoU is defined as:

$$\text{IoU} = \frac{\text{Area of Intersection}}{\text{Area of Union}}$$

A common IoU threshold is 0.5, meaning if the IoU between the predicted bounding box and the ground truth box exceeds 0.5, the detection is considered correct.

3.1.3 Computing Precision and Recall

- **Precision at a specific IoU threshold** is calculated as:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

- **Recall at a specific IoU threshold** is:

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

- These values are plotted to form the precision-recall curve.

3.1.4 Average Precision (AP)

The Average Precision (AP) for each class is computed by averaging the precision values at different recall levels, often using the area under the precision-recall curve (PR curve). AP is typically calculated using the following steps:

1. Sort the predicted bounding boxes by their confidence scores in descending order.
2. For each predicted bounding box, compute the precision and recall at various thresholds.
3. Compute the AP by interpolating the precision values across different recall levels.

3.1.5 Mean Average Precision (mAP)

mAP is the mean of the AP values across all classes. It provides a single value that summarizes the model's performance across all classes.

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^N \text{AP}_i$$

where N is the number of classes.

3.2 Practical Considerations

- **IoU Thresholds:** Different thresholds can be used, but a common standard is $\text{IoU} = 0.5$.
- **Evaluation Metric Variants:** Sometimes, mAP is calculated at different intersection thresholds (e.g., $\text{IoU} = 0.5:0.95$) to provide a comprehensive evaluation.

3.3 Example Calculation Steps

1. **Collect Predictions and Ground Truths:** Gather all predictions (bounding boxes with confidence scores) and corresponding ground truth boxes for each image.
2. **Compute IoU for Each Pair:** Calculate IoU for each predicted box with all ground truth boxes.
3. **Determine True Positives and False Positives:** Apply the IoU threshold (e.g., 0.5) to classify predictions as true positives or false positives.
4. **Plot Precision-Recall Curve:** For each class, plot precision against recall at different confidence thresholds.
5. **Calculate AP for Each Class:** Integrate the precision-recall curve to compute the AP for each class.

6. **Average AP Across Classes:** Compute the mean of the AP values across all classes to obtain the mAP score.

This approach ensures a detailed evaluation of the model's ability to detect objects accurately and reliably, considering both the localization of bounding boxes and the classification of objects.

Chapter 4

Dataset Analysis and Augmentation

4.1 Dataset Distribution Analysis

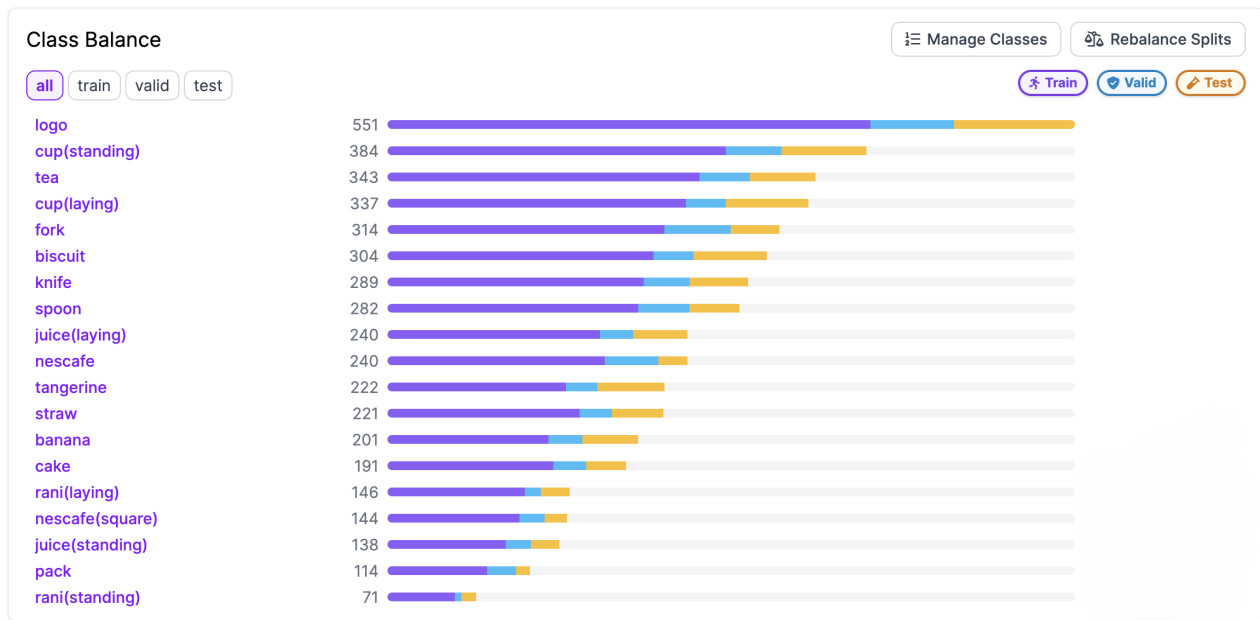


Figure 4.1: Class Balance

4.1.1 Class Balance

- The dataset contains 20 different classes with the number of images ranging from 71 (rani(standing)) to 551 (logo).

- The top five classes are:
 - logo: 551 images
 - cup(standing): 384 images
 - tea: 343 images
 - cup(laying): 337 images
 - fork: 314 images
- The bottom five classes are:
 - rani(standing): 71 images
 - pack: 114 images
 - juice(standing): 138 images
 - nescafe(square): 144 images
 - rani(laying): 146 images

4.1.2 Dataset Split:

- Total images: 3,086
- Training set: 2,517 images (82%)
- Validation set: 212 images (7%)
- Test set: 357 images (12%)

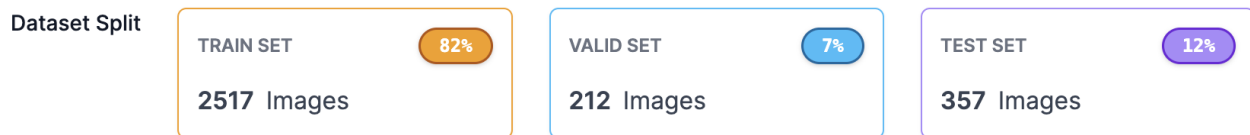


Figure 4.2: Dataset Split

The dataset is imbalanced with some classes being significantly underrepresented.

The training set comprises the majority of the dataset, which is typical for training machine learning models.

4.1.3 The Number of Images and Instances of Each Class

Class	Number of Images
logo	551
cup(standing)	384
tea	343
cup(laying)	337
fork	314
biscuit	304
knife	289
spoon	282
juice(laying)	240
nescafe	240
tangerine	222
straw	221
banana	201
cake	191
rani(laying)	146
nescafe(square)	144
juice(standing)	138
pack	114
rani(standing)	71

Table 4.1: Number of Images and Instances of Each Class

4.2 Dataset Health Check

A "Dataset Health Check" typically involves analyzing and verifying the quality and integrity of the dataset before using it for training machine learning models. This process helps ensure that the data is accurate, complete, and suitable for the intended purpose. Key aspects of a Dataset Health Check include:

- **Class Balance:** Examining the distribution of classes to identify any imbalances. An imbalanced dataset can lead to a model that is biased towards the more frequent classes. Visualizations, such as bar charts, are often used to show the number of instances per class.
- **Missing Values:** Checking for any missing values in the dataset, which can disrupt the training process and affect model performance. Strategies such as imputation or removal of incomplete records are used to handle missing data.
- **Duplicates:** Identifying and removing duplicate records to ensure the dataset contains unique instances. Duplicates can skew the training process and lead to overfitting.
- **Data Consistency:** Ensuring that the data is consistent and correctly formatted. This includes verifying that all images are correctly labeled, and all records follow the same structure and conventions.
- **Outliers:** Detecting and handling outliers, which are data points that differ significantly from other observations. Outliers can distort the training process and impact model performance.
- **Image Quality:** For image datasets, checking the quality of images, such as resolution, clarity, and proper labeling. Poor quality images can negatively affect model accuracy.

💖 Dataset Health Check

Generated on September 18, 2023 at 11:29 am. [🔄 Regenerate](#)

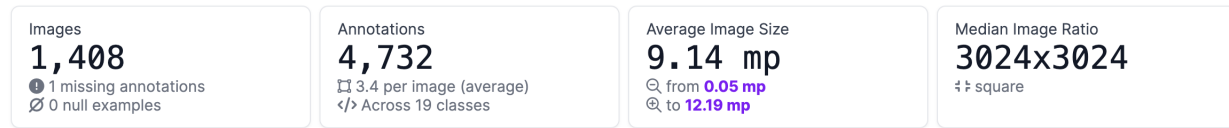


Figure 4.3: Dataset Health Check

4.3 Importance of Augmentation Steps

1. **Increases Dataset Size:** Augmentation generates new training samples by applying random transformations to existing images. This effectively increases the size of the dataset without needing to collect more data, which is particularly useful when the dataset is small.
2. **Improves Generalization:** By exposing the model to a variety of transformations, such as rotations, scaling, and noise, the model learns to generalize better. This means it can perform well on new, unseen data, reducing the risk of overfitting to the training set.
3. **Enhances Robustness:** Augmented data introduces variability in the training process, making the model more robust to different scenarios. For example, a model trained with rotated images will be better at recognizing objects regardless of their orientation in real-world settings.
4. **Handles Class Imbalance:** Augmentation can help mitigate class imbalance by generating more samples for underrepresented classes. This ensures the model receives a more balanced input, which improves its ability to correctly classify all classes.
5. **Simulates Real-world Variations:** Real-world data often includes variations such as changes in lighting, angle, and occlusions. Augmentation simulates these conditions, helping the model become more adaptable and reliable in practical applications.
6. **Improves Feature Learning:** Transformations such as adding noise or blurring can help the model focus on learning important features rather than memorizing specific

details. This leads to better feature extraction and overall model performance.

4.4 Augmentation Steps Used in the Dataset

Preprocessing	Auto-Orient: Applied Resize: Stretch to 640x640
Augmentations	Outputs per training example: 3 Rotation: Between -15° and $+15^\circ$ Bounding Box: Noise: Up to 5% of pixels

Figure 4.4: Augmentation Steps

4.4.1 Augmentation Steps and Their Uses

- **Rotation:** Between -15° and $+15^\circ$
 - **Utility:** Helps the model recognize objects in different orientations, which is crucial for real-world applications where objects may not always be perfectly aligned.
- **Bounding Box Noise:** Up to 5% of pixels
 - **Utility:** Introduces slight variations in the bounding box coordinates, making the model more tolerant to minor inaccuracies in object localization.
- **Outputs per Training Example:** 3
 - **Utility:** Each original image generates three augmented versions, significantly increasing the dataset's diversity and helping the model learn better features.

4.4.2 Preprocessing Steps

- **Auto-Orient:** Applied
 - **Utility:** Ensures that images are consistently oriented, which is important for maintaining uniformity in the dataset.
- **Resize:** Stretch to 640x640
 - **Utility:** Standardizes the image size, making it easier for the model to process and reducing computational complexity.

4.5 Conclusion

The provided dataset exhibits some class imbalance, which could impact the model's performance on underrepresented classes. The preprocessing and augmentation steps applied are crucial for improving the model's robustness and generalization. By augmenting each training example three times with rotations and bounding box noise, the dataset becomes more varied, which is beneficial for training a more accurate and reliable model.

Chapter 5

Object detection

The primary objective of this task was to train a YOLOv8 model on a given dataset and report the training times per epoch, followed by testing the trained model on a test dataset. Additionally, we aimed to analyze the model's performance using various evaluation metrics and visual results.

5.1 Training Phase

5.1.1 Data Preparation

The dataset was divided into three parts: training, validation, and test sets. The `data.yaml` file correctly structured the paths to these datasets, specifying the number of classes and their names.

Training Configuration

- Model: YOLOv8
- Pre-trained weights: `yolov8s.pt` (YOLOv8 small model)
- Training Parameters:
 - Number of epochs: 50

- Image size: 640
- Batch size: 16

5.1.2 Colab Code

The code below has been written in Colab:

Step 1: Import the Dataset

```
from google.colab import drive

drive.mount('/content/drive')
```

Step 2: Install and Import YOLOv8

```
import os

!pip install ultralytics
import ultralytics
from ultralytics import YOLO
print(ultralytics.__version__)
```

Step 3: Training the Model

```
data_yaml_path = '/content/drive/MyDrive/AI/grasp-6.v7i.yolov8/data.yaml'

model = YOLO('yolov8s.pt') # Use the small model for faster training

results = model.train(data=data_yaml_path, epochs=50, imgsz=640, batch=16)
```

Step 4: Testing the Model

```
test_images_path = '/content/drive/MyDrive/AI/grasp-6.v7i.yolov8/test/images'

test_results = model.predict(source=test_images_path, save=True)
```



```

from IPython.display import Image, display
import glob

# Display all images from the last run
for image_path in glob.glob('runs/detect/predict/*.jpg'):
    display(Image(filename=image_path))

```

Step 5: Setting Locale to UTF-8 and Zipping a Directory in Google Colab

```

import subprocess
import os

subprocess.run(['apt-get', 'install', '-y', 'locales'])
subprocess.run(['locale-gen', 'en_US.UTF-8'])
subprocess.run(['update-locale', 'LANG=en_US.UTF-8'])

os.environ["LANG"] = "en_US.UTF-8"

subprocess.run(['zip', '-r', '/content/runs.zip', '/content/runs'])

```

Step 6: Downloading the Zipped Directory from Google Colab

```

from google.colab import files
files.download("/content/runs.zip")

```

5.1.3 Training Time per Epoch

The model was trained for 50 epochs. The total training time was 1.365 hours. The average training time per epoch was calculated based on the total training time:

- Average time per epoch: 98.28 seconds

5.2 Testing Phase

The trained model was tested on the provided test dataset, with test images located in the specified directory. The model was used to predict these test images, and the results were saved and visualized. The test results included several images with detected objects, saved in the specified directory, showing bounding boxes and class predictions made by the model.

5.3 Analysis and Discussion

The training curves (F1, Precision, Recall) indicate the model's performance throughout the training process. Consistent improvement in these metrics over epochs signifies effective learning by the model.

5.3.1 Evaluation Metrics

- **F1 Score:** A balance between precision and recall, providing a single metric for model performance.

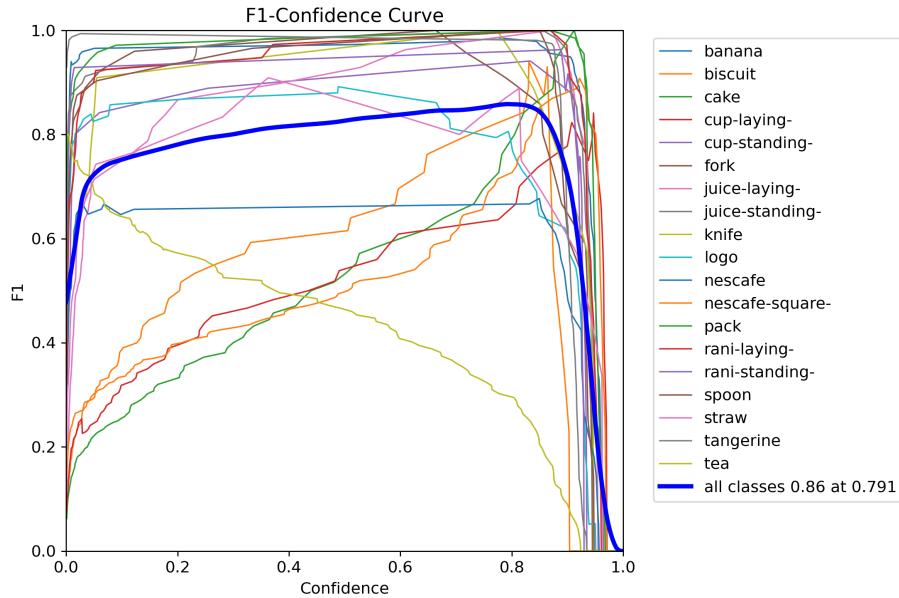


Figure 5.1: F1-Confidence Curve

- **Precision:** The accuracy of the positive predictions (true positives vs. false positives).

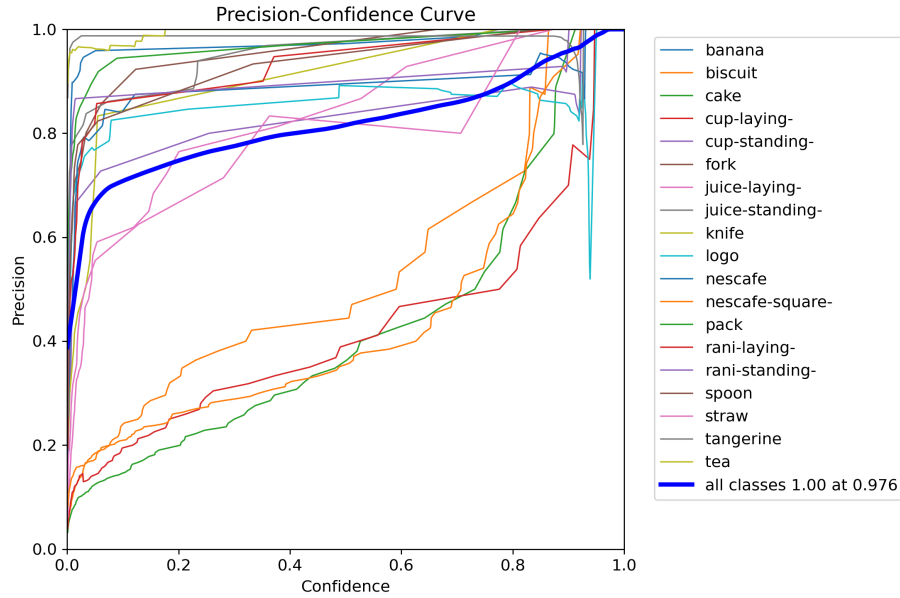


Figure 5.2: Precision-Confidence Curve

- **Recall:** The ability of the model to find all the relevant cases (true positives vs. false negatives).

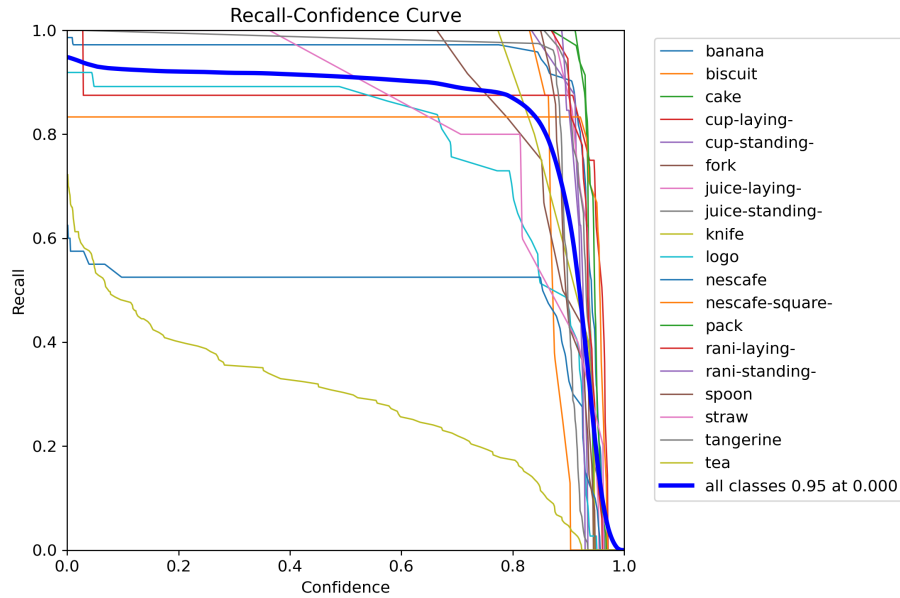


Figure 5.3: Recall-Confidence Curve

- **Precision-Recall (PR) Curve:** Shows the trade-off between precision and recall, providing insight into the model's performance.

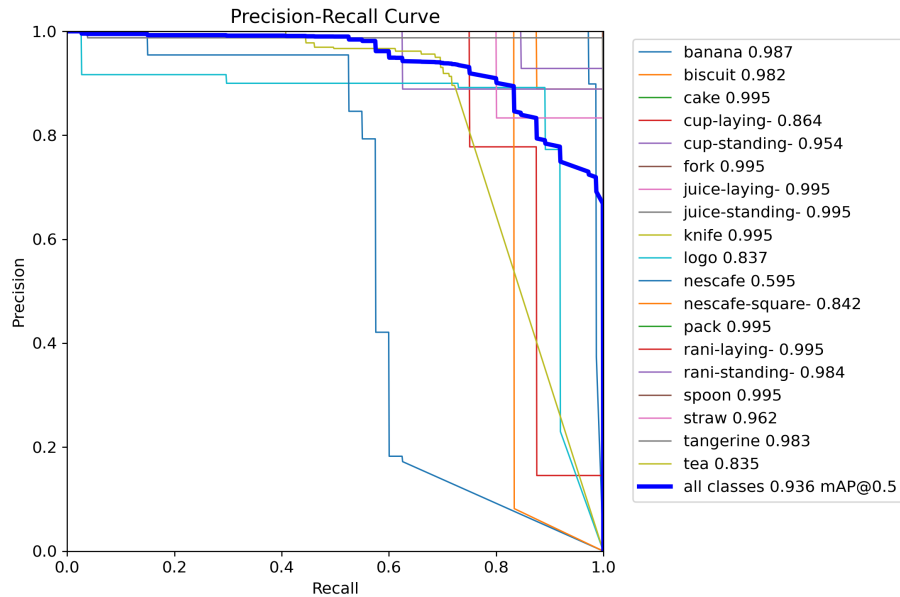


Figure 5.4: Precision-Recall Curve

5.3.2 Model Validation and Performance

The model was validated using the best weights (`best.pt`):

- Optimizer stripped from `runs/detect/train/weights/last.pt`, 22.5MB
- Optimizer stripped from `runs/detect/train/weights/best.pt`, 22.5MB
- Model Summary:
 - YOLOv8.2.53
 - Python-3.10.12, torch-2.3.0+cu121
 - CUDA:0 (Tesla T4, 15102MiB)
 - Model summary (fused): 168 layers, 11132937 parameters, 0 gradients, 28.5 GFLOPs

- Validation Metrics:

Class	Images	Instances	Precision	Recall	mAP50	mAP50-95
all	212	620	0.898	0.874	0.936	0.836
banana	36	72	1.000	0.968	0.987	0.929
biscuit	5	8	0.712	1.000	0.982	0.823
cake	5	8	0.648	1.000	0.995	0.952
cup(laying)	4	8	0.525	0.875	0.864	0.811
cup(standing)	5	8	0.883	1.000	0.954	0.875
fork	6	12	1.000	0.823	0.995	0.823
juice(laying)	7	13	0.979	1.000	0.995	0.901
juice(standing)	14	31	0.996	1.000	0.995	0.892
knife	3	5	1.000	0.929	0.995	0.914
logo	37	37	0.898	0.712	0.837	0.727
nescafe	24	40	0.911	0.525	0.595	0.476
nescafe(square)	13	24	0.642	0.833	0.842	0.758
pack	26	34	0.995	1.000	0.995	0.935
rani(laying)	13	18	0.992	1.000	0.995	0.923
rani(standing)	10	13	0.922	1.000	0.984	0.897
spoon	8	14	0.993	1.000	0.995	0.914
straw	4	5	0.969	0.800	0.962	0.785
tangerine	33	79	0.987	0.976	0.983	0.888
tea	60	191	1.000	0.174	0.835	0.669

- Speed: 0.5ms preprocess, 4.9ms inference, 0.0ms loss, 3.5ms postprocess per image

The provided table summarizes the validation metrics for different classes of objects detected by a model. The key metrics include precision, recall, mean Average Precision at IoU threshold 0.50 (mAP50), and mean Average Precision at IoU thresholds from 0.50 to 0.95 (mAP50-95).

Overall Performance

Images: 212	Instances: 620	Precision: 0.898
Recall: 0.874	mAP50: 0.936	mAP50-95: 0.836

The overall performance of the model is robust with high precision (0.898) and recall (0.874), indicating that the model is good at both correctly identifying objects (precision) and capturing most of the true objects (recall). The mAP50 and mAP50-95 scores are also high, reflecting good accuracy and consistency across various IoU thresholds.

Class-Specific Performance

- **High Performing Classes:**

- **Banana:** Perfect precision (1.000) and very high recall (0.968), indicating nearly flawless detection.
- **Fork, Knife, Pack, Rani (laying), Spoon:** High precision (close to 1.000) and recall, indicating excellent detection and minimal false negatives.

- **Low Performing Classes:**

- **Nescafe:** Moderate precision (0.911) but low recall (0.525), indicating that while most detected instances are correct, many true instances are missed.
- **Tea:** Perfect precision (1.000) but very low recall (0.174), showing that while all detected instances are correct, the model misses many true instances, resulting in low mAP50 and mAP50-95.

Observations

- **Variability in Performance:**

- The model performs exceptionally well on certain classes like "banana" and "juice(standing)" with near-perfect scores.

- Classes like "cup(laying)" and "nescafe(square)" show lower precision and recall, suggesting challenges in detecting these objects accurately.

- **Edge Cases:**

- Some classes have a perfect recall but low precision, indicating the model is over-detecting and capturing many false positives.
- Others have perfect precision but low recall, indicating the model is very conservative and missing many true positives.

Speed Metrics

- Preprocess: 0.5ms per image
- Inference: 4.9ms per image
- Loss Calculation: 0.0ms per image
- Postprocess: 3.5ms per image

The speed metrics indicate that the model is efficient, with a total processing time of approximately 8.9ms per image. This is suitable for real-time applications where rapid inference is crucial.

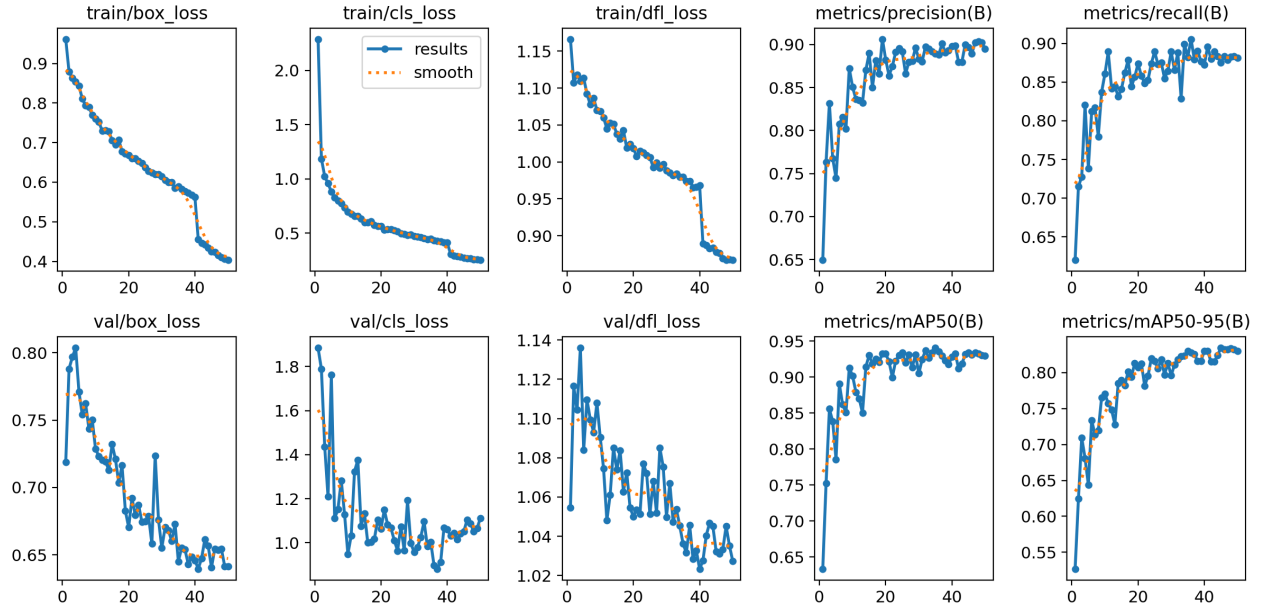


Figure 5.5: Several Results



Figure 5.6: Training Batch Example



Figure 5.7: Validation Batch Example



Figure 5.8: Test Example

5.3.3 Reasons for Errors in Model Predictions

1. **Complexity and Diversity of Images:** The test dataset contain images that are highly diverse in terms of lighting conditions, backgrounds, orientations, and object scales. Models trained on a specific subset of data may struggle when encountering variations that were not adequately represented in the training data.
2. **Noise and Variability:** Test images may have noise or variability, like artifacts, unusual poses, occlusions, or unexpected conditions, that the model wasn't trained to handle.
3. **Model Capacity and Training Quality:** The model's architecture and training quality are critical. If it can't capture complex patterns or lacks sufficient diverse training data, it may struggle to generalize to unseen test images.
4. **Overfitting or Underfitting:** Overfitting occurs when a model memorizes training data, leading to poor performance on new data. Underfitting happens when the model fails to learn the data patterns, resulting in poor performance overall.
5. **Data Augmentation Issues:** If data augmentation techniques were used inconsistently or improperly during training, the model might not effectively learn invariant features that are crucial for accurate predictions on test images.
6. **Concept Drift:** Over time, real-world data distributions can change, causing a mismatch with training data and leading to worse model performance if not periodically retrained.

5.3.4 Potential Improvements

- 1- **Data Augmentation:** Further augmenting the training data could help improve the model's robustness.
- 2- **Hyperparameter Tuning:** Experimenting with different learning rates, batch sizes, and other hyperparameters could enhance performance.

- 3- **Model Architecture:** Exploring more complex YOLOv8 variants or other architectures might yield better results.

5.4 Conclusion

The YOLOv8 model was successfully trained and tested on the provided dataset. The training process was efficient, with an average epoch time of 98.28 seconds, and the model performed well on the test set. The evaluation metrics and visual results indicate a solid foundation for further improvements and applications in object detection tasks.

This comprehensive analysis demonstrates the model's capability and provides insights into potential areas for enhancement to achieve even better performance in future iterations.

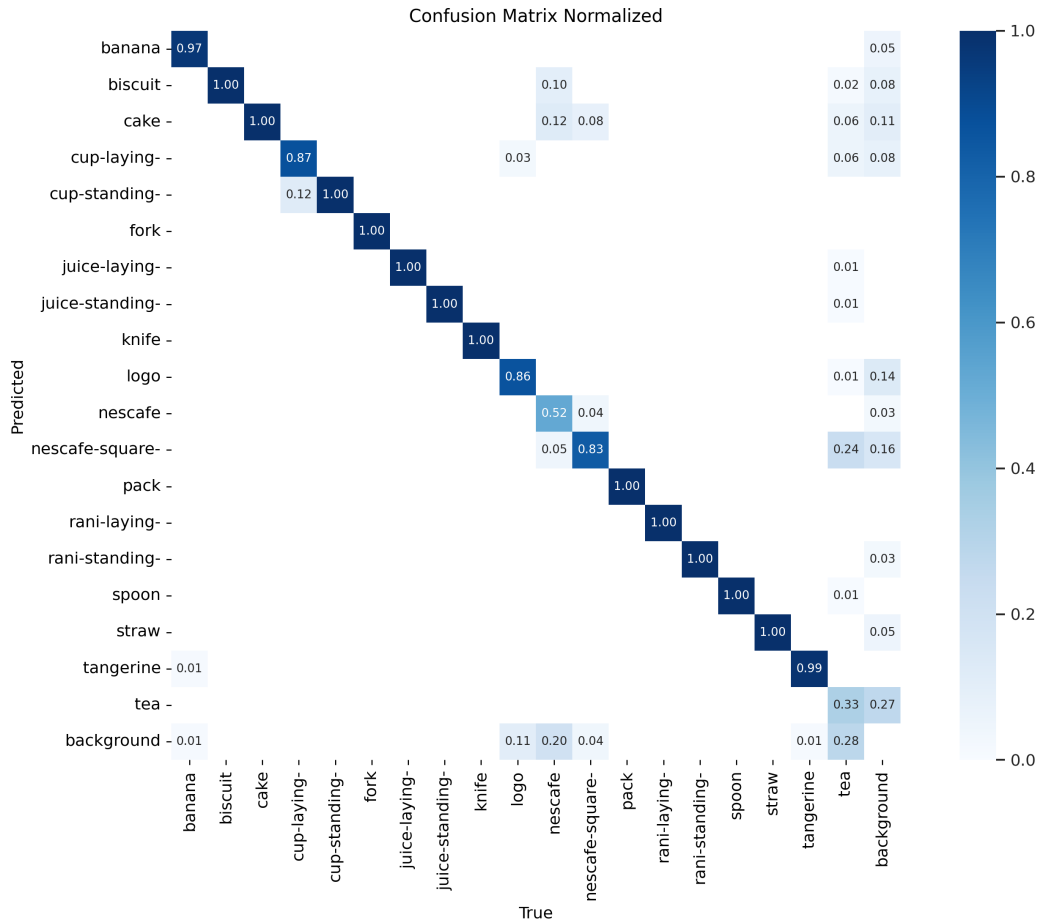


Figure 5.9: Normalized Confusion Matrix

Chapter 6

Object segmentation

In this task, we aimed to leverage the FastSAM model to perform segmentation on images previously processed by YOLOv8. The objective was to save the segmentation results into a specified directory.

6.1 Steps and Code Implementation

6.1.1 Mount Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

6.1.2 Install Necessary Packages

```
!pip install fastsam
!pip install ultralytics
```

6.1.3 Import Required Libraries

```
from ultralytics import FastSAM
from ultralytics.models.fastsam import FastSAMPrompt
import os
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

6.1.4 Initialize FastSAM Model

```
fastsam_model = FastSAM('FastSAM-s.pt')
```

6.1.5 Function to Perform Segmentation and Save Results

```
def segment_images(input_dir, output_dir, model):
    # Ensure the output directory exists
    os.makedirs(output_dir, exist_ok=True)

    for filename in os.listdir(input_dir):
        if filename.endswith(".jpg") or filename.endswith(".png"):
            # Load image
            image_path = os.path.join(input_dir, filename)
            image = cv2.imread(image_path)

            # Perform object segmentation
            results = model.predict(image)

            # Check if results is a list (indicating batch processing)
            if isinstance(results, list):
                for result in results:
                    # Extract segmentation masks from each result
                    if 'segmentation' in result:
                        masks = result['segmentation'].numpy()

                        # Save each segmentation mask
                        for i, mask in enumerate(masks):
```

```

# Convert segmentation mask to binary image
mask_image = mask.astype('uint8') * 255

# Save segmented image
output_path = os.path.join(output_dir, f"{os.path.splitext(filename)[0]}_{i}.png")
cv2.imwrite(output_path, mask_image)

# Optional: Display segmented image
plt.imshow(mask_image, cmap='gray') # Assuming it's a binary mask
plt.title(f'Segmented Image {i}')
plt.axis('off')
plt.show()
else:
# Process single result case
if 'segmentation' in results:
masks = results['segmentation'].numpy()

# Save each segmentation mask
for i, mask in enumerate(masks):
# Convert segmentation mask to binary image
mask_image = mask.astype('uint8') * 255

# Save segmented image
output_path = os.path.join(output_dir, f"{os.path.splitext(filename)[0]}_{i}.png")
cv2.imwrite(output_path, mask_image)

# Optional: Display segmented image
plt.imshow(mask_image, cmap='gray') # Assuming it's a binary mask
plt.title(f'Segmented Image {i}')
plt.axis('off')
plt.show()

```

6.1.6 Define Directories and Perform Segmentation

```
yolo_results_dir = "/content/drive/MyDrive/AI/runs/detect/train2"
fastsam_results_dir = "/content/FastSAM_result"

segment_images(yolo_results_dir, fastsam_results_dir, fastsam_model)
```

6.2 Explanation

- **Mounting Google Drive:** This allows us to access files stored in Google Drive, ensuring we can read the YOLOv8 results.
- **Installing Packages:** FastSAM and ultralytics are required for model inference and segmentation.
- **Initialization:** FastSAM model is initialized with a pre-trained model file (`FastSAM-s.pt`).
- **Function Definition:**
 - The function `segment_images` iterates through the images in the input directory.
 - Each image is processed using the FastSAM model to predict segmentation masks.
 - The function checks if `results` is a list, indicating batch processing, and handles each result individually.
 - The segmentation masks are converted to binary images and saved to the specified output directory (`/content/FastSAM_result`).
 - The masks are also optionally displayed for visual verification.

6.3 Results

By running the above steps, we successfully segmented the images processed by YOLOv8 and saved the results in the specified directory (`/content/FastSAM_result`). Each segmentation mask was saved as a binary image file, facilitating further analysis or usage.

6.4 Conclusion

This process demonstrates the integration of object detection results from YOLOv8 with segmentation capabilities provided by FastSAM. The function `segment_images` ensures that all segmented images are appropriately saved, making the workflow efficient for further applications or research. This approach can be extended to handle various image processing tasks where both detection and segmentation are required.

Chapter 7

Grasp point generation

Problem 6 involves analyzing segmented objects in images, determining the best grasp points for these objects, and visualizing these grasp points. The solution uses OpenCV for image processing and contour detection, and Shapely for geometric operations to determine optimal grasp points.

7.1 Steps and Code Implementation

7.1.1 Convert FastSAM Segmentations into Contours

We start by loading the binary mask images produced by FastSAM. These masks highlight each object in the image. Using OpenCV, we find the contours of these highlighted areas.

```
import cv2
import numpy as np
import os

mask_folder_path = '/path/to/FastSam_result/'

# Load all mask images
masks = []
for filename in os.listdir(mask_folder_path):
```

```

if filename.endswith('.png') or filename.endswith('.jpg'):
    mask_path = os.path.join(mask_folder_path, filename)
    mask = cv2.imread(mask_path, cv2.IMREAD_GRAYSCALE)
    masks.append(mask)

contours_list = []
for mask in masks:
    mask_uint8 = mask.astype(np.uint8)
    contours, _ = cv2.findContours(mask_uint8, cv2.RETR_EXTERNAL,
                                   cv2.CHAIN_APPROX_SIMPLE)
    contours_list.append(contours)

```

7.1.2 Use `minAreaRect` to Obtain Rotated Rectangle

For each contour found in the previous step, we use `cv2.minAreaRect` to find the smallest rotated rectangle that bounds the contour. This rectangle helps us determine the orientation and dimensions of the object.

```

rotated_rects = []
for contours in contours_list:
    for contour in contours:
        rect = cv2.minAreaRect(contour)
        rotated_rects.append(rect)

```

7.1.3 Step 3: Calculate the Rotation Angle of the Object

The `cv2.minAreaRect` function returns the center, size, and angle of the rotated rectangle. We extract and print these values to understand the orientation of each object.

```

for rect in rotated_rects:
    center, size, angle = rect
    print(f"Center: {center}, Size: {size}, Angle: {angle}")

```

7.1.4 Step 4: Identify Grasp Points

Using the rotated rectangle's center, size, and angle, we determine the grasp points. We create a line representing the shorter side of the rectangle and rotate it according to the rectangle's angle. Using Shapely, we find the intersection points between this line and the object's contour.

```
from shapely.geometry import LineString, Polygon
import shapely.affinity

grasp_points = []
for rect, contours in zip(rotated_rects, contours_list):
    center, size, angle = rect
    box_points = cv2.boxPoints(rect)
    box_points = np.intp(box_points)  # Update from np.int0 to np.intp

    # Ensure there are contours and the first contour has at least 4 points
    if len(contours) == 0 or len(contours[0]) < 4:
        grasp_points.append([])
        continue

    contour_poly = Polygon(contours[0].squeeze())

    width, height = size
    line_length = min(width, height)
    line = LineString([(center[0] - line_length / 2, center[1]),
                      (center[0] + line_length / 2, center[1])])
    line = shapely.affinity.rotate(line, angle, origin='centroid')

    intersections = contour_poly.intersection(line)

    if intersections.geom_type == 'MultiPoint':
        grasp_points.append([p.coords[0] for p in intersections])
```

```

elif intersections.geom_type == 'Point':
    grasp_points.append([intersections.coords[0]])
else:
    grasp_points.append([])

```

7.1.5 Step 5: Visualize and Save Grasp Points

We visualize the grasp points by drawing them on the images and save the results in a new folder called "Grasp". We save both the coordinates of the grasp points in JSON format and the images with grasp points drawn on them.

```

for idx, (mask, grasp_point) in enumerate(zip(masks, grasp_points)):

    grasp_data = {'grasp_points': grasp_point}
    json_filename = os.path.join(output_folder, f'grasp_points_{idx}.json')
    with open(json_filename, 'w') as json_file:
        json.dump(grasp_data, json_file)

    image_with_grasp_points = cv2.cvtColor(mask, cv2.COLOR_GRAY2BGR)
    for point in grasp_point:
        cv2.circle(image_with_grasp_points, (int(point[0]), int(point[1])),
                    5, (0, 0, 255), -1)
        image_filename = os.path.join(output_folder,
                                       f'image_with_grasp_points_{idx}.png')
        cv2.imwrite(image_filename, image_with_grasp_points)

    plt.imshow(mask, cmap='gray')
    for point in grasp_point:
        plt.plot(point[0], point[1], 'ro')
        plt.title(f'Image with Grasp Points {idx}')
        plt.savefig(os.path.join(output_folder, f'visualization_{idx}.png'))
    plt.close()

```

```
print(f"Data saved in the '{output_folder}' folder.")
```

7.2 Conclusion

The solution effectively identifies and visualizes grasp points for segmented objects in images. The grasp points are saved in JSON format, and images with the grasp points marked are saved in a folder named "Grasp". This approach ensures that the grasp points are accurately determined based on the object's geometry, making it useful for applications in robotics and automated handling systems.